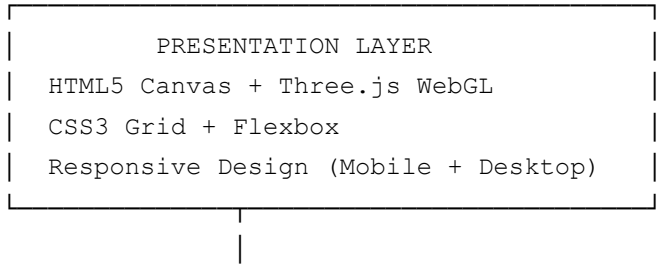
BONEPILE - Development Guide

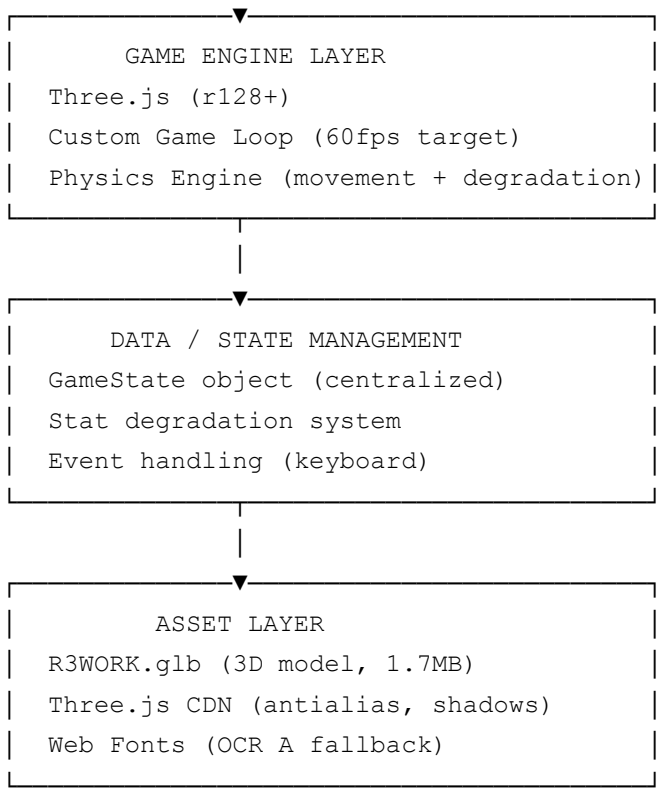
Internal Reference: `BONEPILE-DEV-v1.0`
Last Updated: 2026-05-16
Maintainer: Carlos Aldair Martinez Cortes (@carmacorte)

📖 Table of Contents

Architecture Overview

Tech Stack





Core Game Loop

```
// Simplified pseudocode
function gameLoop() {
    // 1. Input Handling
    processKeyboardInput();  // A/D, Space, Shift, E, R

    // 2. Update Simulation
    updatePlayerPosition();
    updateAnimationState();
    updateStatDegradation();
    applyAbilities();

    // 3. Apply Physics
    clampWorldBounds();
    updateModelTransform();

    // 4. Render
    renderer.render(scene, camera);

    // 5. UI Update
    updateHUD();
    updateDebug();

    requestAnimationFrame(gameLoop);
```

```
}
```

Development Setup

Local Environment

```
# Clone repository
git clone https://github.com/carmacorte/bonepile.git
cd bonepile

# Start local server (Python 3)
python -m http.server 8000

# Open in browser
# http://localhost:8000

# Or use Node.js http-server
npm install -g http-server
http-server
```

Dependencies

- **Three.js (r128+)** - CDN link (no npm required)

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/three.js/r128/three.min.js
<script src="https://cdnjs.cloudflare.com/ajax/libs/three.js/r128/loaders/GLTF
```

- **Assets**
  - `R3WORK.glb` - 3D model (glTF 2.0)
  - `index.html` - Game entry point
  - `README.md` - Documentation

Browser Compatibility

Browser	Support	Notes
Chrome 90+	✅ Full	Best performance
Firefox 88+	✅ Full	Excellent WebGL

Safari 15+	✅ Full	Good on macOS/iOS
Edge 90+	✅ Full	Chromium-based
Mobile Safari	⚠️ Partial	Optimized, may be slower
IE 11	❌ Not supported	No WebGL 2.0

Code Organization

Current File Structure

```
bonepile/
├── index.html           # Main game file (~450 lines)
├── R3WORK.glb           # 3D model asset
├── README.md            # User documentation
├── DEVELOPMENT.md       # This file
└── .gitignore
```

Future Modular Structure (v2.0)

```
bonepile/
├── index.html
├── src/
│   ├── game.js          # Game state & loop
│   ├── input.js         # Keyboard handling
│   ├── physics.js       # Movement & degradation
│   ├── render.js        # Three.js setup
│   ├── ui.js            # HUD updates
│   └── abilities.js     # Special actions
├── assets/
│   ├── models/
│   │   └── R3WORK.glb
│   ├── sounds/
│   ├── textures/
│   └── shaders/
├── styles/
│   └── main.css
├── README.md
└── DEVELOPMENT.md
```

Systems & Mechanics

Stat System

Health (INTEGRIDAD)

- **Range:** 0-100%
- **Degradation:** -0.01 per frame when WALK, -0.03 per frame when DASH
- **Recovery:** Repair ability (+30%, cost 15 FLUX)
- **Game Over:** Reaches 0%

Signal (SEÑAL)

- **Range:** 0-100%
- **Recovery:** +0.01 per frame (passive)
- **Mechanic:** Represents connectivity with Functionalist network
- **Future:** Blocked by IPC jamming in certain zones

AOI Risk (RIESGO AOI)

- **Range:** 0-100%
- **Increase:** +0.005/frame during WALK, +0.02/frame during DASH
- **Decrease:** -0.05/frame during HIDE
- **Mechanic:** Probability of detection by optical scanners
- **Game Over:** Reaches 100% (captured)

Flux (FLUX)

- **Range:** 0-100%
- **Consumption:**
  - DASH: -25 per use
  - HIDE: -5 per frame
  - REPAIR: -15 per use

- **Recovery:** Pickup items (future)
- **Resource:** Energy for sabotage & survival

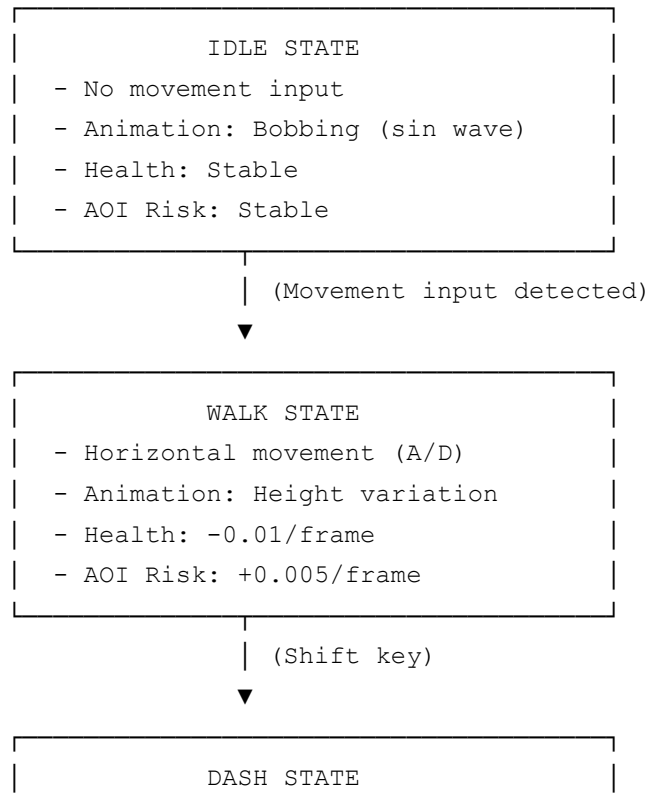
Movement System

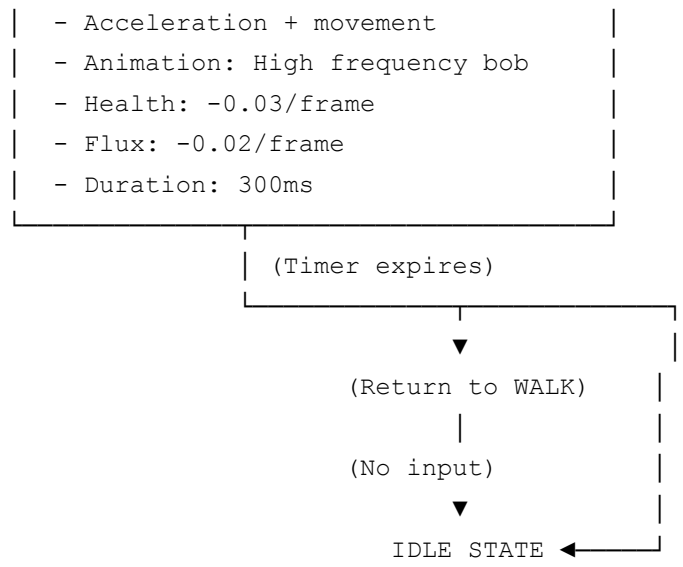
```
// Simplified movement code
const speed = 0.05;  // Units per frame
const worldBounds = { min: -8, max: 8 };

if (keys['a'] || keys['arrowleft']) {
    playerVX = -speed;
    direction = -1;
}
if (keys['d'] || keys['arrowright']) {
    playerVX = speed;
    direction = 1;
}

playerX += playerVX;
playerX = Math.max(worldBounds.min, Math.min(worldBounds.max, playerX));
model.position.x = playerX;
```

State Machine






```
Parallel State: HIDE (toggle with E)
  - Reduces model scale (0.7x)
  - Lowers position (-0.3 Y)
  - Decreases AOI Risk (-0.05/frame)
  - Consumes Flux continuously
```

3D Model Integration

Importing R3WORK.glb

Three.js GLTFLoader Flow:

```
const loader = new THREE.GLTFLoader();

loader.load('R3WORK.glb', (gltf) => {
    // gltf.scene = Root THREE.Group
    // gltf.scenes = Array of scenes
    // gltf.animations = Array of animation clips
    // gltf.asset = Metadata

    const model = gltf.scene;

    // Setup shadows
    model.traverse((node) => {
        if (node.isMesh) {
            node.castShadow = true;
            node.receiveShadow = true;
        }
    });
```

```
    scene.add(model);
});
```

Model Properties (Current)

Property	Value	Notes
Format	glTF 2.0 Binary	.glb extension
Scale	1:1	Fits in ~0.5m bounding box
Poly Count	~50k-100k	(estimate)
Materials	PBR (Metallic/Roughness)	Proper shading
Bones	Present	Ready for animation
File Size	1.7MB	Optimized

Model Optimization Opportunities

```
// Potential improvements:
1. Bake textures (reduce material count)
2. LOD (Level of Detail) variants
3. Instancing for multiple units
4. Draco compression (glTF)
5. Basis universal textures
```

Animations (Future)

When R3WORK.glb animations are needed:

```
const mixer = new THREE.AnimationMixer(model);

// For each animation clip:
gltf.animations.forEach((clip) => {
    const action = mixer.clipAction(clip);
    action.play();
});

// In game loop:
const deltaTime = clock.getDelta();
mixer.update(deltaTime);
```

Animation System

Current Animations (Procedural)

Idle Animation

```
if (state === 'IDLE') {
    // Vertical bobbing using sine wave
    model.position.y = Math.sin(Date.now() * 0.001) * 0.1;
    // Frequency: 0.001 rad/ms = 6.28 rad/s = 1 cycle/second
    // Amplitude: ±0.1 units
}
```

Walk Animation

```
if (state === 'WALK') {
    animTime += 0.05;  // Increment each frame

    // Height variation during walk
    model.position.y = Math.sin(animTime) * 0.15;
    // Pins would bob with legs (future: skeletal animation)
}
```

Direction Flip

```
// Binary rotation based on direction
model.rotation.y = direction === 1 ? 0 : Math.PI;
// No smooth interpolation (instant 180° flip)
// Future: Smooth transition over 0.2s
```

Animation Improvement Roadmap

V1.1 - Smooth rotations + frame-based walk cycle

```
const targetRotY = direction === 1 ? 0 : Math.PI;
model.rotation.y += (targetRotY - model.rotation.y) * 0.1;  // Lerp
```

V1.2 - Skeletal animation from glTF

```
// Use native animation clips from R3WORK.glb
// Play "walk_cycle" based on velocity
```

V1.3 - Procedural leg articulation

```
// Calculate pin bend angles based on walk phase
const legBend = Math.sin(animTime) * Math.PI / 4;  // ±45°
pins.forEach((pin, i) => {
    pin.rotation.z = i % 2 === 0 ? legBend : -legBend;
});
```

UI & HUD

Current HUD Layout

```
┌──────────────────────────────────────────────────────┐
│ INTEGRIDAD | SEÑAL | RIESGO AOI | ESTADO | FLUX     │
│ [████▒▒▒▒] | [...] | [▒▒██▒▒▒▒] | MÓVIL  | [...▒▒] │
│ 80%        | 75%   | 35%        |         | 45%      │
└──────────────────────────────────────────────────────┘
```

CSS Grid Implementation

```
#hud {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(120px, 1fr));
    gap: 15px;
}

/* Responsive: Mobile */
@media (max-width: 768px) {
    grid-template-columns: repeat(3, 1fr);
}
```

HUD Update Logic

```
function updateHUD() {
    // Write stat values
    document.getElementById('integrityValue').textContent =
        Math.round(gameState.health) + '%';
```

```
    // Update bar widths (CSS)
    document.getElementById('integrityFill').style.width =
        gameState.health + '%';

    // Color-coded bars
    // Green (health), Cyan (signal), Orange (AOI), Pink (flux)
}
```

Future HUD Enhancements

- **Mini-map** - Isometric view of world + player position
- **Threat indicator** - Visual/audio warning when AOI risk > 70%
- **Flux gauge** - Pulsing glow when flux low
- **Event log** - Scrolling notifications (“Detected!” “Hidden!”)
- **Ability cooldown** - Visual indicator for DASH/REPAIR recharge

---

Performance Optimization

Current Performance Targets

Metric	Target	Notes
Frame Rate	60 FPS	Desktop/laptop
Mobile	30-40 FPS	Acceptable
Load Time	<2s	R3WORK.glb + Three.js
Memory	<100MB	Typical browser limit
Model VRAM	~50MB	GPU memory

Optimization Techniques

1. Rendering Optimization

```
// Frustum culling (Three.js automatic)
// Occlusion culling (future: multi-zone levels)
```

```
// LOD (Level of Detail) models for far objects
// Batch rendering (instancing)

// Current: Single model, no batching needed
```

2. Memory Management

```
// Reuse objects instead of creating new ones
const stats = { health: 0, signal: 0 };  // Object pool

// Avoid creating objects in game loop
// Bad:
for (let i = 0; i < 10; i++) {
    const pos = new THREE.Vector3(...);  // Creates 10 objects/frame!
}

// Good:
const tempVec = new THREE.Vector3();
for (let i = 0; i < 10; i++) {
    tempVec.set(...);  // Reuse same object
}
```

3. Physics Simplification

```
// Current: Only position X updates
// No gravity/collision (intentional simplicity)
// No raycast checks

// Future: Only raycasts when necessary
if (gameState.hidden && gameState.aoiRisk < 50) {
    checkForNearbyEnemies();  // Conditional check
}
```

Profiling Tools

```
// In DevTools Console:
1. Performance tab (Frame analysis)
2. console.time('label') / console.timeEnd('label')
3. Three.js stats:
   const stats = new Stats();
   document.body.appendChild(stats.dom);
```

Testing & Debugging

Debug Mode

Enable by opening DevTools console:

```
// Display debug info (top-right corner)
// Shows: Frame count, Position, State, Health, Flux
```

Manual Testing Checklist

```
☐ Keyboard input (A/D, Space, Shift, E, R)
  ☐ Movement left/right
  ☐ Jump animation
  ☐ Dash effect + flux consumption
  ☐ Hide toggle + AOI reduction
  ☐ Repair + health/flux

☐ Animation states
  ☐ IDLE: Smooth bobbing
  ☐ WALK: Height variation
  ☐ DASH: Fast pulsing
  ☐ HIDE: Scale reduction

☐ HUD updates
  ☐ Health bar moves realistically
  ☐ Signal recovers passively
  ☐ AOI increases with movement
  ☐ Flux decreases with abilities

☐ Edge cases
  ☐ Stats clamped (0-100%)
  ☐ Position clamped (world bounds)
  ☐ Abilities fail when insufficient resources
  ☐ Hidden state cancels on movement?

☐ Performance
  ☐ 60 FPS on desktop
  ☐ No lag spikes during ability use
  ☐ Smooth transitions
```

Common Issues & Solutions

Issue	Cause	Solution
Model not loading	Wrong path	Check browser Network tab, verify R3WORK.glb in same dir
Black screen	Camera issue	Check camera.position + lookAt
No shadows	Shadow map disabled	Set renderer.shadowMap.enabled = true
Jittery movement	Frame rate inconsistent	Use deltaTime-based movement
HUD overlaps	CSS grid overflow	Use max-width on debug element

---

Roadmap

V1.0 ✅ (Current - Vertical Slice)

- R3WORK 3D model loading
- Basic movement (A/D)
- State machine (IDLE/WALK/DASH/HIDE)
- Stat degradation system
- Ability system (DASH, HIDE, REPAIR)
- HUD with real-time stats
- Isometric camera
- GitHub Pages deployment

V1.1 🔄 (Next - Polish & Feedback)

- Smooth model rotation (lerp)
- Particle effects (Dash, Hide)
- Sound design (footsteps, flux hiss)
- Mobile touch controls
- Difficulty settings (health degradation rate)

- Game over screen (health = 0 or AOI = 100)

V1.2 (Level Design & Gameplay)

- Multiple levels/zones
- Procedural PCB layout generator
- Repair station pickups (restore health)
- Flux vial pickups
- Hazards (electrical traces, heat zones)
- Simple puzzle elements

V1.3 (Enemies & Combat)

- AOI scanner (moving threat)
- IPC patrol drones (simple AI)
- Detection system (noise, movement detection)
- Chase/evasion mechanics
- Boss encounter (Regulador 78XX)

V1.4 (Narrative & Polish)

- Dialog system (NPC Functionalists)
- Quest framework
- Branching endings
- Intro/outro cinematics
- Full sound design

V2.0 (Major Expansion)

- Full campaign (4-6 levels)
- Upgrade system (improve stats)
- Multiplayer? (peer-to-peer)
- Custom level editor
- Mod support

Contributing Guidelines

Code Style

```
// camelCase for variables/functions
const playerHealth = 100;
function updateStats() {}

// PascalCase for classes/constructors
class GameState {}
const player = new Player();

// Comments for non-obvious logic
// Degradation formula: health -= movement_distance * 0.01
if (Math.abs(vx) > 0.1) {
    health -= 0.01;
}
```

Commit Messages

```
[FEATURE] Add particle effects to dash ability
[FIX] Correct HUD stat bar overflow on mobile
[PERF] Optimize model rendering with LOD system
[DOCS] Update README with v1.1 changelog
```

Pull Request Process

1. Fork repository
2. Create feature branch: `git checkout -b feature/dash-particles`
3. Implement + test locally
4. Commit with clear messages
5. Push to branch
6. Open PR with description of changes

References & Resources

Three.js Documentation

- Three.js Official Docs
- GLTFLoader Guide
- Shadow Maps

Game Design

- Game Programming Patterns
- State Machines
- Event Systems

Performance

- Web Performance APIs
- Frame Budget

Manufacturing Reference

- IPC-A-610 Standard (Electronics Assembly)
- PCB Manufacturing Flow

Contact & Support

Issues & Bugs: GitHub Issues (carmacorte/bonepile)
Feature Requests: GitHub Discussions
Questions: @carmacorte
Website: SMTintel.dev

> **"Code is the map of the territory. Make it beautiful."** — SMTintel Design Philosophy

---